

@azdocuments

PYTHON PROGRAMMING

SUBJECT CODE: 1BPLC105B/205B

MODULE-4

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

PYTHON PROGRAMMING

1BPLC105B/205B

Module-4

Modules: Random numbers, the time module, the math module, creating your own modules, Namespaces,

Scope and lookup rules, Attributes and the dot Operator, Three import statement variants.

Mutable versus immutable and aliasing

Object oriented programming: Classes and Objects — The Basics, Attributes, Adding methods to our class,

Instances as arguments and parameters, Converting an instance to a string, Instances as return values.

Chapter: 8.1-8.8, 9.1, 11.1

4.1 Python Modules and the random Module

A **module** in Python is a file that contains Python code such as functions, variables, and classes, which can be reused in other Python programs. Python comes with many **built-in modules** as part of its **standard library**, which help programmers perform common tasks easily. Examples of such modules are the **turtle module** (used for graphics) and the **string module** (used for string operations). Python also provides a **help system** that allows users to explore all available standard modules and understand how to use them.

Random Numbers in Python (random Module)

In many programs, we need **random numbers**. Some common uses of random numbers are:

- Simulating dice throws, coin tosses, or lottery games
- Shuffling playing cards
- Generating random positions in games (e.g., enemy spaceships)
- Simulating natural events like rainfall
- Supporting security features such as encryption

Python provides the **random module** to handle all such tasks.

Creating a Random Number Generator

```
import random

rng = random.Random()
```

Here, rng acts like a **black box** that generates random values.

Generating Random Integers (randrange)

```
dice_throw = rng.randrange(1, 7)
```

- This generates a random integer from **1 to 6**.

- The **lower limit is included**, and the **upper limit is excluded**.
- All values have **equal probability** (uniform distribution).

You can also specify a **step value**:

random_odd = rng.randrange(1, 100, 2)

- This generates a **random odd number less than 100**.

Generating Random Floating-Point Numbers (random)

delay_in_seconds = rng.random() * 5.0

- rng.random() returns a floating-point number in the range **[0.0, 1.0)**.
- This means **0.0 is possible**, but **1.0 is not included**.
- Multiplying by 5.0 scales the value to the range **[0.0, 5.0)**.
- These values are also **uniformly distributed**.

Other Distributions

Apart from uniform distribution, the random module can generate values following other distributions, such as:

- **Normal (bell-shaped) distribution**, useful for modeling rainfall, medical data, or scientific measurements.

Shuffling a List (shuffle)

cards = list(range(52))

rng.shuffle(cards)

- This represents a deck of **52 cards**.
- The shuffle() method randomly rearranges the elements of the list.
- Note: shuffle() works only on **lists**, not directly on range objects.

4.1.1 Repeatability and Testing (Python Random Numbers)

Random number generators in Python are **not truly random**. They are based on a **deterministic algorithm**, meaning that if the same starting conditions are used, the generator will produce the **same sequence of numbers every time**. Because of this behavior, they are called **pseudo-random number generators**.

Seed Value

A random number generator starts with a **seed value**.

- The seed determines the **initial state** of the generator.
- Every time a random number is generated, the internal state (seed) is updated.
- Future random numbers depend on the **current state** of the generator.

Why Repeatability Is Important

Repeatability is especially useful for:

- **Debugging programs**

- **Writing unit tests**
- **Verifying program behavior**

When testing, we want a program to behave **the same way every time it runs**. Random behavior can make debugging difficult, so Python allows us to **fix the seed**.

Creating a Generator with a Fixed Seed

```
import random

drng = random.Random(123)
```

- Here, 123 is the **explicit seed value**.
- Every time this program is run, drng will generate **exactly the same sequence of random numbers**.
- This ensures **repeatable and predictable results**.

Default Behavior (Without a Seed)

If no seed is provided:

```
random.Random()
```

- Python usually uses the **current system time** as the seed.
- This results in **different random sequences each time the program runs**.

Important Practical Note

- **Fixed seed** → useful for **testing and debugging**
- **Variable seed (time-based)** → useful for **games and real-world simulations**

For example:

- In testing: fixed seed is desirable
- In games: fixed shuffling would make the game boring and predictable

4.1.2 Picking Balls from Bags, Throwing Dice, Shuffling Cards

This section explains how Python's random module can be used to:

- Generate random numbers
- Handle **duplicates vs non-duplicates**
- Model real-world probability situations such as dice throws, lotteries, and card shuffling

Case 1: Random Numbers With Duplicates (With Replacement)

Function Explanation

```
import random

def make_random_ints(num, lower_bound, upper_bound):
    """
```

```
    Generate a list containing num random ints between lower_bound
    and upper_bound. upper_bound is an open bound.
    """
    rng = random.Random()
    result = []
    for i in range(num):
        result.append(rng.randrange(lower_bound, upper_bound))
    return result
```

Example Output

```
make_random_ints(5, 1, 13)
# Output: [8, 1, 8, 5, 6]
```

Key Observations

- **Duplicates are allowed** (e.g., 8 appears twice).
- This behavior is expected in scenarios like:
  - Throwing a die multiple times
  - Tossing a coin
- In statistics, this is called **sampling with replacement**:
  - After drawing a value, it is "put back," so it can appear again.

Case 2: Random Numbers Without Duplicates (Without Replacement – Small Range)

If duplicates are **not allowed**, the previous algorithm is incorrect.

Correct Approach: Shuffle and Slice

```
xs = list(range(1, 13))   # Numbers 1 to 12 (months)
rng = random.Random()
rng.shuffle(xs)
result = xs[:5]
```

Explanation

- First, create a list of all possible values.
- Shuffle the list randomly.
- Take only the first n elements.
- This guarantees **no duplicates**.

Real-World Example

- Picking lottery numbers

- Drawing cards from a deck
- Selecting unique months

In statistics, this is called **sampling without replacement**.

Problem with Shuffle-and-Slice for Large Ranges

If the range is **very large**, this method becomes inefficient.

Example:

- Pick 5 numbers between 1 and 10,000,000 (without duplicates)
- Creating and shuffling a list of 10 million numbers is a **performance disaster**.

Case 3: Random Numbers Without Duplicates (Large Range – Efficient Method)

Improved Algorithm

```
import random
def make_random_ints_no_dups(num, lower_bound, upper_bound):
    """
    Generate a list containing num random ints between
    lower_bound and upper_bound. upper_bound is an open bound.
    The result list cannot contain duplicates.
    """
    result = []
    rng = random.Random()
    for i in range(num):
        while True:
            candidate = rng.randrange(lower_bound, upper_bound)
            if candidate not in result:
                break
        result.append(candidate)
    return result
```

Example Output

```
xs = make_random_ints_no_dups(5, 1, 10000000)
print(xs)
# [3344629, 1735163, 9433892, 1081511, 4923270]
```

Why This Works

- Random numbers are generated one by one.
- A number is accepted **only if it is not already in the list**.
- Efficient when:
    - The number of required values is small
    - The range is very large

The Pitfall (Very Important for Exams)

Problem Case

xs = make_random_ints_no_dups(10, 1, 6)

What Will Happen?

- Numbers possible: 1, 2, 3, 4, 5 → only **5 unique values**
- Requested numbers: **10**
- The program will enter an **infinite loop**
- Eventually, it will **never terminate**

Why?

Because it is impossible to generate **more unique numbers than the range allows**.

4.2 The time Module

As programs become larger and algorithms more complex, an important question arises:
Is our code efficient?

One practical way to answer this is by **measuring execution time**—that is, how long a piece of code takes to run. Python provides the **time module** for this purpose.

The clock() Function (as per the given text)

The time.clock() function returns a **floating-point value** representing the number of **seconds elapsed since the program started running**.

Basic Timing Strategy

1. Call clock() **before** the code you want to measure → store in t0
2. Execute the code
3. Call clock() **after** execution → store in t1
4. Compute elapsed time as:
5. t1 − t0

This difference tells us **how fast the code ran**.

Example: Comparing Two Ways to Sum a List

Python has a built-in sum() function. We can also write our own summation logic. This example compares the **performance** of both approaches.

User-Defined Sum Function

```
import time
def do_my_sum(xs):
    sum = 0
    for v in xs:
        sum += v
    return sum
```

Test Setup

```
sz = 10000000          # 10 million elements
testdata = range(sz)
```

Timing the User-Defined Function

```
t0 = time.clock()
my_result = do_my_sum(testdata)
t1 = time.clock()
print("my_result = {0} (time taken = {1:.4f} seconds)"
      .format(my_result, t1 - t0))
```

Timing the Built-in sum() Function

```
t2 = time.clock()
their_result = sum(testdata)
t3 = time.clock()
print("their_result = {0} (time taken = {1:.4f} seconds)"
      .format(their_result, t3 - t2))
```

Sample Output

```
my_result    = 49999995000000 (time taken = 1.5567 seconds)
their_result = 49999995000000 (time taken = 0.9897 seconds)
```

Analysis of Results

- Both methods produce the **same correct result**
- The **built-in sum() function is faster**
- The user-defined function is approximately **57% slower**
- Built-in functions are optimized at a **lower level**, making them more efficient

Despite this, summing **10 million numbers in under a second** using the built-in function is very efficient.

Important Note (Modern Python)

In newer versions of Python, time.clock() is deprecated. Functions like:

- time.perf_counter()
- time.process_time()

4.3 The math Module

The **math module** in Python provides mathematical functions and constants similar to those found on a scientific calculator. It is commonly used for **trigonometry, logarithms, square roots**, and other mathematical computations.

To use the math module, it must first be imported:

import math

Mathematical Constants

The math module provides important mathematical constants:

math.pi

- Value of π (pi)
- Output: 3.141592653589793

math.e

- Value of **e**, the base of natural logarithms
- Output: 2.718281828459045

Common Mathematical Functions

Square Root

math.sqrt(2.0)

- Returns the square root of 2
- Output: 1.4142135623730951

Trigonometric Functions and Angle Measurement

In Python (and most programming languages), **angles are measured in radians**, not degrees.

Converting Degrees to Radians

math.radians(90)

- Converts 90 degrees into radians
- Output: 1.5707963267948966

Sine Function

math.sin(math.radians(90))

- Calculates sine of 90 degrees
- Output: 1.0

Inverse Trigonometric Function

math.asin(1.0) * 2

- asin(1.0) returns π/2
- Multiplying by 2 gives π
- Output: 3.141592653589793

Radians vs Degrees

- **Radians** are the standard unit for angles in programming.
- Python provides:
  - math.radians() → degrees to radians
  - math.degrees() → radians to degrees

Important Conceptual Difference

math Module vs random and turtle Modules

- In **random** and **turtle**, we:
  - Create objects
  - Call methods on those objects
  - Objects have **state**
    - Example:
      - Turtle has position, color, direction
      - Random generator has a seed
- In the **math module**:
  - Functions are **pure functions**
  - They do **not depend on any state or history**
  - The same input always gives the same output
  - Example:

- math.sqrt(2.0) will always return the same value

Therefore, math functions are **not methods of an object**. They are simply **standalone functions grouped inside the math module**.

4.4 Creating Your Own Modules

In Python, we can create **our own modules** by simply saving a Python script with a **.py file extension**. Any file that contains Python code (functions, variables, etc.) and is saved with .py automatically becomes a module.

Example: Creating a Custom Module

Suppose we create a file named **seqtools.py** with the following code:

```
def remove_at(pos, seq):
    return seq[:pos] + seq[pos+1:]
```

What This Function Does

- Removes the element at position pos from a sequence
- Works with strings, lists, or any sequence that supports slicing

Using the Custom Module

Once the file is saved, we can use it in another Python program or in the Python interpreter by importing it.

```
import seqtools
```

Example Usage

```
s = "A string!"
seqtools.remove_at(4, s)
```

Output

```
'A sting!'
```

- The character at index 4 (the letter **r**) is removed
- The function works exactly like a built-in utility

Important Rules About Importing Modules

- **Do not include .py** in the import statement
  ```
  import seqtools
  import seqtools.py
  ```
- Python automatically looks for files ending in .py
- The module file must be:
  - In the **same directory**, or
  - In a directory listed in Python's module search path

Why Use Modules?

Using modules helps in:

- Breaking **large programs into smaller, manageable parts**
- Reusing code across multiple programs
- Organizing related functions together
- Improving **readability and maintenance**
- A module is created by saving code in a **.py file**
- Functions inside a module can be accessed using:
- module_name.function_name()
- The .py extension is **not written** during import
- Modules support **code reuse and better program structure**

4.5 Namespaces

A **namespace** is a collection of **identifiers (names)** such as variables, functions, and objects that belong to a **module**, **function**, or **class**. Namespaces help organize code and prevent **name conflicts**.

Generally, a namespace contains **related items**, for example:

- All mathematical functions in the math module
- All random-related functions in the random module

Why Namespaces Are Important

Namespaces allow:

- The **same identifier name** to be used in different places
- Multiple programmers to work on the same project **without naming collisions**
- Better organization and readability of large programs

Each module has its **own namespace**, so identical names in different modules do not interfere with each other.

Example: Module Namespaces

module1.py

question = "What is the meaning of Life, the Universe, and Everything?"

answer = 42

module2.py

question = "What is your quest?"

answer = "To seek the holy grail."

Using Both Modules Together

import module1

import module2

```
print(module1.question)
print(module2.question)
print(module1.answer)
print(module2.answer)
```

Output

```
What is the meaning of Life, the Universe, and Everything?
What is your quest?
42
To seek the holy grail.
```

Explanation

- Both modules define question and answer
- No conflict occurs because:
  - module1.question and module2.question exist in **different namespaces**

Function Namespaces

Functions also create their **own namespaces**.

Example

```
def f():
    n = 7
    print("printing n inside of f:", n)
def g():
    n = 42
    print("printing n inside of g:", n)
n = 11
print("printing n before calling f:", n)
f()
print("printing n after calling f:", n)
g()
print("printing n after calling g:", n)
```

Output

```
printing n before calling f: 11
printing n inside of f: 7
```

printing n after calling f: 11

printing n inside of g: 42

printing n after calling g: 11

Explanation

- There are **three different variables named n**
- Each n exists in a **different namespace**:
  - Global namespace
  - Function f() namespace
  - Function g() namespace
- They do **not collide**, even though the names are the same

This is similar to having multiple people named "Bruce" — the name is the same, but the individuals are different.

Relationship Between Namespaces, Files, and Modules

Python follows a **one-to-one mapping**:

- **One file → One module → One namespace**

Example:

- File name: math.py
- Module name: math
- Namespace name: math

So when you write:

import math

math.sqrt(4)

You are accessing sqrt inside the **math namespace**.

Important Conceptual Clarification

- **Files and directories**:
  - Concerned with **where code is stored** on the computer
- **Modules and namespaces**:
  - Concerned with **how code is organized logically**

In Python, these concepts appear tightly connected, but this is **not true in all languages**.

Key Warning (Very Important for Understanding)

If you rename a file in Python:

- The **module name changes**
- All import statements must be updated
- All references to that namespace must also change

Other programming languages (like C#) may:

- Allow one module across multiple files
- Allow multiple namespaces in one file
- Share one namespace across many files

Therefore, **do not confuse file structure with namespaces**, even though Python blends them.

4.6 Scope and Lookup Rules

The **scope** of an identifier (variable, function name, etc.) is the **region of the program where that identifier can be accessed or used**.

Python determines which identifier to use by following well-defined **scope and lookup rules**.

The Three Important Scopes in Python

1. Local Scope

- Identifiers declared **inside a function**
- Stored in the **function's namespace**
- Each function has its **own local namespace**
- Exists only during function execution

2. Global Scope

- Identifiers declared **at the top level of a module (file)**
- Accessible throughout the module
- Shared by all functions in that module

3. Built-in Scope

- Identifiers provided by Python itself
- Examples: range, min, len, print
- Available **without importing anything**

Inspecting Scopes in Python

Python provides built-in functions to examine scopes:

- locals() → shows local namespace
- globals() → shows global namespace
- dir() → shows names in a namespace

Scope Lookup Precedence Rules

When Python encounters an identifier, it searches scopes in this order:

1. **Local scope**
2. **Global scope**
3. **Built-in scope**

The **innermost scope always takes precedence**.

Example 1: Hiding a Built-in Name

```
def range(n):
    return 123 * n
print(range(10))
```

Output

1230

Explanation

- Python finds range in the **global scope**
- This hides the built-in range
- Therefore, Python calls the **user-defined function**, not the built-in one Redefining built-in names like range or min is **bad practice** and should be avoided because it causes confusion.

Example 2: Local vs Global Variables

```
n = 10
m = 3
def f(n):
    m = 7
    return 2 * n + m
print(f(5), n, m)
```

Output

17 10 3

Step-by-Step Explanation

- n = 10 and m = 3 are in the **global namespace**
- When f(5) is called:
  - A new **local n** is created with value 5
  - A new **local m** is created with value 7
- Inside the function:
- 2 * 5 + 7 = 17
- After the function returns:
  - Global n and m remain **unchanged**
  - The local variables are destroyed

Scope of Identifiers in the Example

- n on line 1 (global):
  - Visible on lines **1, 2, 6, and 7**
  - Hidden on lines **3, 4, and 5** by the local n
- f:
  - Created by def
  - Exists in the **global namespace**
  - Can be called anywhere in the module after its definition

4.7 Attributes and the Dot Operator

In Python, **variables defined inside a module** are called **attributes of that module**. Similarly, **objects** (such as modules, functions, and classes) can also have attributes.

Attributes in Python

Module Attributes

- Any variable or function defined inside a module becomes an **attribute** of that module.
- These attributes belong to the module's **namespace**.

Example:

module1.py

question = "What is the meaning of life?"

Here, question is an **attribute** of module1.

Object Attributes

Objects can also have attributes. Examples include:

- __doc__ → documentation string of an object

- __annotations__ → type annotations of a function

These attributes store information related to the object.

The Dot Operator (.)

The **dot operator** is used to **access attributes** of a module or object.

Accessing Module Attributes

module1.question

module2.question

Even though both modules contain an attribute named question, there is **no conflict** because each belongs to a different module namespace.

Accessing Functions Using the Dot Operator

Modules can contain **functions as well as variables**, and both are accessed using the dot operator.

Example:

seqtools.remove_at

- seqtools → module name
- remove_at → function inside the module

This expression refers to the remove_at function defined in the seqtools module.

Fully Qualified Names

When a name includes the module (or object) it belongs to, it is called a **fully qualified name**.

Examples:

- module1.question
- module2.answer
- seqtools.remove_at

Using fully qualified names:

- Clearly specifies **which attribute** is being referred to
- Avoids ambiguity when multiple modules have attributes with the same name

4.8 Three Import Statement Variants

Python provides **three main ways** to import modules or names from modules into the current namespace. Each method affects **how names are accessed** and **where they are placed**.

Method 1: Import the Entire Module (Recommended)

import math

x = math.sqrt(10)

Explanation

- Only the name math is added to the current namespace.
- Functions and constants inside the module must be accessed using **dot notation**.
- Example:
    - math.sqrt(10)
    - math.pi

Advantages

- Clear and explicit
- Avoids name conflicts
- Easy to understand which module a function comes from

This is the **preferred and safest method**.

Method 2: Import Specific Names from a Module

from math import cos, sin, sqrt

x = sqrt(10)

Explanation

- The names cos, sin, and sqrt are imported **directly** into the current namespace.
- The module name math is **not imported**.
- Using math.sqrt(10) will cause an error.

Advantages

- Less typing
- Convenient when using a few functions frequently

Disadvantages

- Possible name clashes
- Less clear where a function originated

Method 3: Import All Names from a Module

from math import *

x = sqrt(10)

Explanation

- All public identifiers from the module are imported into the current namespace.
- Functions can be used **without qualification**.

Disadvantages (Important for Exams)

- High risk of name conflicts
- Makes code harder to read and debug
- Not recommended in professional code

Using an Alias (Shorthand Import)

```
import math as m
m.pi
```

Explanation

- The module is imported with an alias (m)
- Reduces typing while maintaining clarity
- Commonly used in practice

Importing Inside a Function (Local Import)

```
def area(radius):
    import math
    return math.pi * radius * radius
x = math.sqrt(10)  # Error
```

Explanation

- math is imported **inside the function**
- It exists only in the **local namespace** of area
- It is **not available globally**
- Therefore, math.sqrt(10) outside the function causes an error

Import Style	Namespace Impact	Qualification Needed	Recommended
import math	Adds math	Yes (math.sqrt)	Yes
from math import sqrt	Adds sqrt	No	Sometimes
from math import *	Adds all names	No	No
import math as m	Adds m	Yes (m.sqrt)	Yes

Introduction to Python Data Types

You have already encountered Python's core data types:

- **bool**
- **int**
- **float**
- **string**
- **tuple**
- **list**
- **dictionary**

In this section, the focus is mainly on **lists and tuples**, and the concepts of **mutability**, **immutability**, and **aliasing**. New data types such as **sets** and **frozensets** are also introduced later.

4.9 Mutable versus Immutable

Mutable Data Types

A **mutable** data type is one whose contents **can be changed after creation**.

Common mutable types:

- **List**
- **Dictionary**

Example: Mutability of Lists

```
my_list = [2, 4, 5, 3, 6, 1]
my_list[0] = 9
print(my_list)
```

Output

```
[9, 4, 5, 3, 6, 1]
```

Here, the value at index 0 is successfully changed, proving that lists are mutable.

Immutable Data Types

An **immutable** data type is one whose contents **cannot be changed after creation**.

Common immutable types:

- **Tuple**
- **String**

Example: Immutability of Tuples

```
my_tuple = (2, 5, 3, 1)
my_tuple[0] = 9
```

Result

TypeError: 'tuple' object does not support item assignment

This error occurs because tuples do not allow modification.

Aliasing

Aliasing occurs when **two or more variables refer to the same object in memory**. This is common with mutable objects like lists.

Example: Aliasing with Lists

```
list_one = [1, 2, 3, 4, 6]

list_two = list_one

list_two[-1] = 5

print(list_one)
```

Output

[1, 2, 3, 4, 5]

Even though only list_two was modified, list_one also changed. This happens because **both variables point to the same list**.

Memory Address Check Using id()

You can verify aliasing using the built-in id() function.

```
id(list_one) == id(list_two)
```

Output

True

This confirms both variables reference the same memory location.

Avoiding Aliasing with Copying

To avoid aliasing, create a **copy** of the list.

Shallow Copy Using Slicing

```
list_one = [1, 2, 3, 4, 6]

list_two = list_one[:]

id(list_one) == id(list_two)
```

Output

False

Now the lists are stored at different memory locations.

```
list_two[-1] = 5
print(list_two)
print(list_one)
```

Output

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 6]
```

Changes to one list do not affect the other.

Limitation: Nested Lists

Shallow copying **does not work correctly for nested lists**, because inner lists are still shared.

Example:

```
a = [[1, 2], [3, 4]]
b = a[:]
b[0][0] = 9
```

Both a and b will reflect the change.

To handle this properly, Python provides the **copy module**, which supports **deep copying**.

4.10 Classes and Objects — the Basics

4.10.1 Object-Oriented Programming

Python is an **object-oriented programming (OOP) language**, which means it provides features that allow programs to be designed using **objects** and **classes**.

What Is Object-Oriented Programming (OOP)?

Object-oriented programming is a programming paradigm that focuses on:

- **Objects** rather than just functions
- Combining **data** and **functionality** into a single unit

OOP originated in the **1960s**, but it became widely adopted in the **mid-1980s** as software systems grew larger and more complex. The main goal of OOP is to:

- Manage **complex software systems**
- Make programs **easier to understand, maintain, and modify**
- Improve **code reuse and scalability**

Procedural Programming vs Object-Oriented Programming

Procedural Programming

- Focuses on **functions or procedures**
- Data and functions are **separate**
- Functions operate on external data

Example idea:

- Write functions
- Pass data to functions
- Data is not tightly bound to behavior

Object-Oriented Programming

- Focuses on **objects**
- An object contains:
  - **Data** (attributes)
  - **Functionality** (methods)
- Data and behavior are **bundled together**

This approach more closely models **real-world systems**.

Objects in Python (Already Encountered)

Even before formally learning OOP, you have already used objects in Python, such as:

- **Turtle objects** (graphics and movement)
- **String objects** (text with built-in methods)
- **Random number generator objects**

Each of these:

- Stores data internally
- Provides methods to operate on that data

Real-World Analogy

In OOP:

- Each **object** represents a real-world entity or concept
- The **data** represents properties
- The **functions (methods)** represent actions

Example (conceptual):

- A *car object*:

- Data: speed, color, fuel
- Methods: start(), stop(), accelerate()

This mirrors how real-world objects behave.

Why Object-Oriented Programming Is Important

OOP helps to:

- Organize large programs into manageable parts
- Reduce complexity
- Improve code readability
- Make programs easier to extend and modify
- Support teamwork by allowing multiple developers to work on different objects

4.10.2 User-Defined Compound Data Types

Python already provides many built-in classes such as int, float, str, and Turtle. In object-oriented programming, we can also **define our own classes** to represent real-world or mathematical concepts. These are called **user-defined compound data types**.

The Concept of a Point

In mathematics, a **point in two dimensions** is represented by two numbers:

- x → horizontal position
- y → vertical position

Examples:

- (0, 0) → origin
- (x, y) → x units to the right and y units up

Common operations on points include:

- Finding the distance from the origin
- Finding the distance between two points
- Computing the midpoint of two points
- Checking whether a point lies inside a shape

To support such operations cleanly, we want to **group the x and y values together**.

Why Not Just Use a Tuple?

A quick solution is to use a tuple like (x, y).
While this works for simple cases, it has limitations:

- No clear names for components (x, y)

- Harder to attach behaviors (methods) to the data

A better and more flexible approach is to **define a new class**.

Defining a Point Class

```
class Point:
    """ Point class represents and manipulates x,y coords. """

    def __init__(self):
        """ Create a new point at the origin """
        self.x = 0
        self.y = 0
```

Understanding the Class Definition

1. class Point:

- Defines a new class named Point
- A class is a **blueprint** for creating objects

2. Docstring

""" Point class represents and manipulates x,y coords. """

- Describes what the class does
- Used by help tools and documentation systems

3. The __init__ Method

def __init__(self):

- Called the **initializer**
- Automatically runs whenever a new object is created
- Used to set initial values of attributes

4. The self Parameter

- Refers to the **newly created object**
- Used to create and access attributes
- self.x and self.y belong to the specific object being created

Creating (Instantiating) Point Objects

```
p = Point()
q = Point()
```

- p and q are two **different Point objects**
- Each has its own x and y attributes

print(p.x, p.y, q.x, q.y)

Output

0 0 0 0

This happens because:

- During initialization, each object's x and y are set to 0

Objects Are Independent

Although p and q are created from the same class:

- They are stored at **different memory locations**
- Changing p.x does not affect q.x

Constructors and Instantiation

- A function like Point() or Turtle() that creates objects is called a **constructor**
- Every class automatically provides a constructor with the **same name as the class**
- The process of:
    1. Creating a new object
    2. Initializing its attributes

is called **instantiation**

Class as a Factory (Important Analogy)

- A **class** is like a factory
- An **object** is a product made by the factory
- Each time the constructor is called:
    - A new object is created
    - The __init__ method sets default values

The class itself is **not an object**—it only contains the instructions for creating objects.

4.10.3 Attributes

Like real-world objects, **object instances in Python have attributes and methods**.
Attributes represent the **data or state** of an object.

Modifying Attributes Using Dot Notation

Attributes of an object instance can be modified using the **dot (.) operator**.

p.x = 3

p.y = 4

- Here, p refers to a Point object
- x and y are **attributes** of that object
- Their values are updated to 3 and 4

Namespaces and Attributes

- Each **instance** has its own **namespace**
- Modules also have namespaces
- Accessing attributes from a module or an object uses the **same syntax**

In this case, we are accessing **data attributes from an object instance**.

Conceptual State of the Object

After the assignments:

- p refers to a Point object
- The object contains:
  - x → 3
  - y → 4
- Each attribute refers to a numeric value

Accessing Attribute Values

Attributes can be accessed using the same dot notation:

print(p.y)

Output

4

Assigning an attribute value to a variable:

x = p.x

print(x)

Output

3

Understanding p.x

The expression:

p.x

means:

“Go to the object that p refers to, and retrieve the value of its attribute x.”

There is **no conflict** between:

- x (a variable in the global namespace)
- p.x (an attribute in the object’s namespace)

The dot notation ensures **unambiguous access**.

Using Attributes in Expressions

Attributes can be used in **any Python expression**.

Example 1: Formatted Output

print("(x={0}, y={1})".format(p.x, p.y))

Output

(x=3, y=4)

Example 2: Calculation Using Attributes

distance_squared_from_origin = p.x * p.x + p.y * p.y

- Calculation:
- $3^2 + 4^2 = 9 + 16 = 25$
- Result:
- distance_squared_from_origin = 25

4.10.4 Improving Our Initializer

Earlier, creating a point at a specific position required **multiple lines of code**:

p = Point()

p.x = 7

p.y = 6

This works, but it is **not convenient** and makes object creation less clean.

Making the Initializer More Flexible

We can improve this by adding **parameters** to the __init__ method so that values can be passed **at the time of object creation**.

Improved Point Class

```
class Point:
    """ Point class represents and manipulates x,y coords. """
def __init__(self, x=0, y=0):
        """ Create a new point at x, y """
        self.x = x
        self.y = y
```

What Changed?

- The __init__ method now accepts two parameters: x and y
- Both parameters have **default values of 0**
- This allows us to:
    - Create points at any location
    - Still create the origin (0, 0) easily

Using the Improved Class

```
p = Point(4, 2)
q = Point(6, 3)
r = Point()      # origin
print(p.x, q.y, r.x)
```

Output

4 3 0

Explanation

- p is created at (4, 2)
- q is created at (6, 3)
- r uses default values and represents (0, 0)

Advantages of This Approach

- Object creation becomes **shorter and clearer**
- Attributes are initialized **immediately**
- Reduces the risk of forgetting to set attributes
- Makes the class easier and safer to use

Important Technical Note (For Understanding)

Strictly speaking:

- The __init__ method does **not create** the object
- The object is created first
- __init__ only **initializes** it with default or provided values

However:

- In practice, **creation and initialization happen together**
- Tools and editors show the __init__ docstring as help when calling the class constructor
- Therefore, the docstring is written to guide the **user of the class**, not to be technically pedantic

4.10.5 Adding Other Methods to Our Class

The real power of using a **class** (like Point) instead of a simple tuple (x, y) becomes clear when we start adding **methods**. A class allows us to group **data** and the **operations that make sense for that data** in one place.

Why Methods Are Important

A tuple such as (6, 7) could represent many things:

- A point (x, y)
- A date (day, month)
- Any other paired data

However:

- Calculating **distance from origin** makes sense only for a **point**
- It does *not* make sense for (day, month) data

By using a class, we ensure that:

- Only **appropriate operations** are allowed
- Data and related behavior stay together
- Each object maintains its **own state**

What Is a Method?

- A **method** is like a function
- It is **defined inside a class**
- It is **called on an object (instance)**
- It is accessed using **dot notation**

Example:

p.distance_from_origin()

This is similar in style to:

```
tess.right(90)
```

Adding a Method to the Point Class

Let's add a method called distance_from_origin to compute the distance of a point from (0, 0).

```
class Point:
    """ Create a new Point, at coordinates x, y """

    def __init__(self, x=0, y=0):
        """ Create a new point at x, y """
        self.x = x
        self.y = y

    def distance_from_origin(self):
        """ Compute my distance from the origin """
        return ((self.x ** 2) + (self.y ** 2)) ** 0.5
```

Using the Method

Example 1

```
p = Point(3, 4)
p.distance_from_origin()
```

Output

5.0

Example 2

```
q = Point(5, 12)
q.distance_from_origin()
```

Output

13.0

Example 3 (Origin)

```
r = Point()
r.distance_from_origin()
```

Output

0.0

Understanding self in Methods

- The **first parameter** of every method refers to the **current object**
- By convention, this parameter is named self
- self.x and self.y refer to the attributes of the object that called the method

Important detail:

- When calling p.distance_from_origin(), we do **not** pass p explicitly
- Python automatically passes the object as the self argument

This happens **behind the scenes**.

Organizational Power of Classes

Using classes provides:

- Better **program structure**
- Logical grouping of **data and operations**
- Improved readability and maintainability
- Clear modeling of real-world concepts

Each instance of the class:

- Has its own attributes
- Can call the same methods
- Behaves independently

4.10.6 Instances as Arguments and Parameters

In Python, **objects (instances)** can be passed to functions **just like any other value**. This is a common and powerful feature of object-oriented programming.

Passing Objects to Functions

We have already seen this concept in earlier examples, such as passing a **turtle object** to a function so that the function can control or manipulate that turtle.

When an object is passed to a function:

- The function receives a **reference** to the object
- The object itself is **not copied**

Important Concept: Aliasing

Variables in Python store **references** to objects, not the objects themselves.

When an object is passed as an argument:

- The parameter inside the function becomes an **alias**
- Both the caller and the function refer to the **same object**
- There is still **only one object in memory**

This is important because:

- Changes made to the object inside the function can affect the original object

Example Using the Point Class

Function Definition

```
def print_point(pt):
    print("({0}, {1})".format(pt.x, pt.y))
```

Explanation

- pt is a parameter that refers to a Point object
- The function accesses the object's attributes using dot notation
- The output format is controlled by the function

Calling the Function

```
p = Point(3, 4)
print_point(p)
```

Output

(3, 4)

What Happens Internally

- p holds a reference to a Point object
- When p is passed to print_point, the parameter pt refers to the **same object**
- No new object is created
- This is why pt.x and pt.y correctly access the attributes of p

Why This Is Useful

Passing instances as arguments allows:

- Functions to operate on different objects dynamically
- Code reuse
- Cleaner and more modular programs

- Separation of responsibilities (functions handle operations, objects store state)

4.10.7 Converting an Instance to a String

When working with classes and objects, it is **not good practice** to write separate "chatterbox" functions that print object details directly. A better object-oriented approach is to let **each object know how to represent itself as a string**.

Initial (Less Preferred) Approach

We may first think of adding a method like this:

```
class Point:
    # ...
    def to_string(self):
        return "({0}, {1})".format(self.x, self.y)
```

Usage:

```
p = Point(3, 4)
print(p.to_string())
```

Output

```
(3, 4)
```

This works, but it is **not ideal**.

Why This Is Not Ideal

Python already provides:

- A built-in **str()** function
- Automatic string conversion inside **print()**

However, without special handling, Python does not know how we want our object to look as a string.

```
str(p)
print(p)
```

Output

```
<__main__.Point object at 0x01F9AA10>
```

This default representation is **not user-friendly**.

The Pythonic Solution: __str__

Python allows us to define a **special method** called __str__.

If a class defines __str__, Python will:

- Automatically use it when str(object) is called
- Automatically use it when print(object) is called

Improved Point Class with __str__

```
class Point:
  # ...

  def __str__(self):
    return "({0}, {1})".format(self.x, self.y)
```

Resulting Behavior

```
p = Point(3, 4)
str(p)
print(p)
```

Output

```
(3, 4)
(3, 4)
```

Now:

- str(p) returns a meaningful string
- print(p) displays the same clean representation

Why __str__ Is Better Than to_string

- Integrates with Python's built-in mechanisms
- Works automatically with print()
- Produces cleaner and more readable output
- Follows object-oriented design principles

4.10.8 Instances as Return Values

In Python, **functions and methods can return object instances**, just like they can return numbers or strings. This is a powerful feature of object-oriented programming, allowing objects to be **created, processed, and returned dynamically**.

Returning an Instance from a Function

Consider the task of finding the **midpoint** between two points.

Function Definition

```
def midpoint(p1, p2):
    """ Return the midpoint of points p1 and p2 """
    mx = (p1.x + p2.x) / 2
    my = (p1.y + p2.y) / 2
    return Point(mx, my)
```

Explanation

- The function receives two Point objects
- It calculates the midpoint coordinates
- It creates and returns a **new Point object**

Using the Function

```
p = Point(3, 4)
q = Point(5, 12)
r = midpoint(p, q)
print(r)
```

Output

```
(4.0, 8.0)
```

The returned value r is a **Point instance**, not a tuple or list.

Returning an Instance from a Method

Instead of a standalone function, we can make this behavior part of the Point class.

Method Definition

```
class Point:
    # ...
 def halfway(self, target):
        """ Return the halfway point between myself and the target """
        mx = (self.x + target.x) / 2
        my = (self.y + target.y) / 2
        return Point(mx, my)
```

Explanation

- self refers to the calling object

- target is another Point object
- The method returns a **new Point instance**

Using the Method

p = Point(3, 4)

q = Point(5, 12)

r = p.halfway(q)

print(r)

Output

(4.0, 8.0)

Function vs Method (Conceptual Difference)

Function	Method
Operates on objects passed as arguments	Operates on the calling object
Defined outside a class	Defined inside a class
midpoint(p, q)	p.halfway(q)

Both approaches return **new objects**, but the method approach is more **object-oriented**.

Composability of Calls

Object creation and method calls can be **combined** without assigning intermediate variables.

Example

print(Point(3, 4).halfway(Point(5, 12)))

Output

(4.0, 8.0)

This works because:

- Point(3, 4) creates an object
- .halfway(Point(5, 12)) returns a new object
- print() displays it using __str__

4.10.9 A Change of Perspective

This section explains an important **conceptual shift** that occurs when moving from **procedural programming** to **object-oriented programming (OOP)**.

Procedural Perspective (Function-Centered)

In procedural programming, the **function is the active agent**.

Example:

print_time(current_time)

This style suggests:

"Hey function print_time, here is an object. Do something with it."

- Functions act on data
- Data is passive
- Responsibility lies with the function

Object-Oriented Perspective (Object-Centered)

In object-oriented programming, the **object is the active agent**.

Example:

current_time.print_time()

This style suggests:

"Hey object current_time, please print yourself."

- Objects contain both **data and behavior**
- Responsibility lies with the object
- Methods belong to the object they operate on

Example from Turtle Graphics

In early turtle examples, we already used this object-oriented style:

tess.forward(100)

This means:

"Hey turtle tess, move yourself forward by 100 units."

The turtle object:

- Knows its own position and direction
- Updates its own state
- Controls its own behavior

Why This Change in Perspective Is Useful

At first, this shift may seem like a stylistic change, but it has real benefits:

- Makes programs **more flexible**

- Improves **code reuse**
- Makes large programs **easier to maintain**
- Reduces dependency between unrelated functions and data

By shifting responsibility from functions to objects:

- Functions become simpler
- Objects manage their own behavior and state

Real-World Analogy (Very Important)

In real life:

- A **microwave oven** has a cook() method
- We do not have a separate cook(microwave) function

Similarly:

- A **cellphone** has methods to:
  - Send an SMS
  - Switch to silent mode
- The behavior is **part of the object itself**

Object-oriented programming mirrors this real-world organization.

Why OOP Fits Human Thinking Better

Humans naturally think in terms of **objects**:

- Objects have properties
- Objects perform actions

OOP matches this mental model by:

- Binding functionality tightly to the object
- Making programs easier to understand and reason about
- of code

4.10.10 Objects Can Have State

One of the most important ideas in object-oriented programming is that **objects can have state**.
The **state of an object** is the collection of data (attributes) that describes the object **at a particular moment in time**.

This state can **change over time** as methods are called on the object.

Object State Explained

An object's state is stored in its **attributes**, and methods are used to:

- **Read** the state
- **Modify** the state

Example 1: Turtle Object

A **turtle object** has a state that includes:

- Position (x, y)
- Heading (direction)
- Color
- Shape

How Methods Affect State

- left(90) → changes the turtle's **heading**
- forward(100) → changes the turtle's **position**
- color("red") → changes the turtle's **color**

Each method updates some part of the turtle's internal state.

Example 2: Bank Account Object

A **bank account object** is another good real-world example.

Possible State of a Bank Account

- Current balance
- Transaction log (history of deposits and payments)

Possible Methods

- get_balance() → checks the current balance
- deposit(amount) → increases the balance
- withdraw(amount, description) → decreases the balance and records the transaction
- show_transactions() → displays the transaction history

Each method:

- Uses the existing state
- Updates the state appropriately

Why State Is Important

Objects become powerful because:

- They **remember information** over time

- Each object maintains its **own independent state**
- Behavior depends on the object's current state

For example:

- Two turtle objects can move differently because they have different positions
- Two bank accounts can have different balances even though they use the same class

Key Concept

- **Attributes = state**
- **Methods = behavior**
- Methods change or use the state of the object
- The object remains consistent and self-contained

Real-World Analogy

Just like real-world objects:

- A mobile phone has state (battery level, silent mode, network)
- Actions like calling or switching to silent **change the phone's state**

OOP mirrors this real-world behavior closely.

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and student-focused resources for every semester.

Connect With Us:

www.azdocuments.in [Click Here]

@azdocuments [Click Here]

Join Our Student Community: [Click Here]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.